



Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Scheduling with Finite Capacity Input Buffers

Nicholas G. Hall, Marc E. Posner, Chris N. Potts,

To cite this article:

Nicholas G. Hall, Marc E. Posner, Chris N. Potts, (1998) Scheduling with Finite Capacity Input Buffers. Operations Research 46(3-supplement-3):S154-S159. <https://doi.org/10.1287/opre.46.3.S154>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

© 1998 INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

SCHEDULING WITH FINITE CAPACITY INPUT BUFFERS

NICHOLAS G. HALL and MARC E. POSNER

The Ohio State University, Columbus, Ohio

CHRIS N. POTTS

University of Southampton, Southampton, England

(Received October 1994; revisions received October 1995, June 1996, November 1996; accepted December 1996)

In many scheduling problems, a newly released job must be stored in an input buffer while it waits to begin processing. The lack of attention given to these buffers in the classical scheduling literature results from the implicit assumption that they have infinite capacity. In modern manufacturing environments, however, there are several important reasons for limiting buffer capacity. We study nonpreemptive single machine dynamic scheduling problems under the assumption that some jobs may be lost, either because of insufficient input buffer capacity, or because due dates cannot be met. The objective is to minimize the weighted or unweighted number of lost jobs. We study problems with zero, fixed or arbitrary buffer capacity, with unit or arbitrary processing times, and with unit or arbitrary buffer storage requirements. We present a complexity classification in which, for each problem, either an efficient algorithm is derived, or a proof is given that such an algorithm is unlikely to exist.

We consider deterministic scheduling problems where decisions involving *input buffers* are important. Consider a situation in which n jobs with varying release dates are to be processed nonpreemptively on a single machine. When a job is released, it can either be processed immediately, stored in an input buffer while it awaits processing, or discarded. The usual assumption is that the input buffer has infinite capacity. However, we assume that the capacity of the input buffer is finite.

Scheduling problems with finite capacity input buffers arise in several manufacturing environments such as production processes with physical space limitations. Finite capacity input buffers model limited warehouse space for components that await processing. In modern manufacturing systems, the provision of buffer space including input buffers is regarded as expensive. Moreover, holding products in a buffer incurs costs due to insurance, record-keeping, and deterioration. Also, the opportunity cost of financing the inventory is often considerable (McClain and Thomas 1985).

A job can be lost if it cannot fit into the buffer or if it cannot complete by its due date. We seek a nonpreemptive schedule that minimizes the total weight (or cost) associated with lost jobs.

Hall et al. (1998) present a computational complexity classification of various single machine scheduling problems with finite capacity *output buffers* that hold completed jobs. For the case of input buffers, Nawijn (1992) studies a constrained problem in which the jobs that are not lost are processed in their order of arrival. Also, Nawijn et al. (1994)

derive a polynomial time algorithm for minimizing the number of lost jobs when the buffer capacity is a constant, and establish that the problem of minimizing the weighted number of lost jobs is intractable. Hall et al. (1997) describe algorithms and complexity results for several preemptive single machine scheduling problems with finite capacity input buffers. Nonpreemptive problems with zero release dates are similarly discussed in Hall et al. (1996).

1. PRELIMINARIES

Let $N = \{1, \dots, n\}$ denote the set of jobs. Let w_j denote the weight, r_j the release date, d_j the due date, p_j the processing time, and s_j the storage requirement of job j , for $j \in N$. Also, let $w_{\max} = \max_{j \in N} \{w_j\}$ and $p_{\max} = \max_{j \in N} \{p_j\}$. The buffer capacity is denoted by b_I . We assume that all w_j , r_j , d_j , p_j , and s_j are integer, and $r_j + p_j \leq d_j$, for $j \in N$. We set $U_j(\sigma) = 0$ if job j completes by its due date in σ ; $U_j(\sigma) = 1$ otherwise. When no ambiguity arises, we simplify $U_j(\sigma)$ to U_j .

We extend the standard three field classification scheme for scheduling problems (Graham et al. 1979) to provide for the classification of scheduling problems with finite capacity input buffers. In the second field, we could have

- $b_I = 0$: input buffer capacity equals zero;
- $b_I = b$: input buffer capacity is a nonzero constant;
- b_I : input buffer capacity is arbitrary (specified as part of problem input).

Table I summarizes our results. A "Yes" in the $d_j = \infty$ column signifies that all due dates are arbitrarily large,

Table I
The Complexity of Input Buffer Problems

Objective	Buffer Type	$d_j = \infty$	$p_j = 1$	$s_j = 1$	Complexity Results	
$\Sigma w_j U_j$	$b_I = 0$	Any	Yes	n.a.	$O(n)$	Thm 2.1
			No	n.a.	$O(n^2)$	Arkin and Silverberg (1987)
	$b_I = b$	Any	No	Any	Binary NPC $O(n^{b+2} \min\{p_{\max}, w_{\max}\})$	
					Nawijn et al. (1994) Thm 2.2	
		Yes	Yes	Yes	$O(n \log n)$	Thm 2.3
				No	$O(n^{b+2})$	Cor 2.1
		No	Yes	Any	$O(n^{b+2})$	
					Cor 2.1	
	b_I	Any	Yes	No	Unary NPC	Thm 2.4
			No	Any	Unary NPC	Thm 2.5
ΣU_j	$b_I = 0$	Any	Yes	n.a.	$O(n)$	Thm 2.1
			No	n.a.	$O(n \log n)$	Thm 3.1
	$b_I = b$	Any	No	Any	$O(n^{b+2})$	
					Cor 3.1	
		Yes	Yes	Any	$O(n \log n)$	
					Thm 3.2	
		No	Yes	Yes	$O(n \log n)$	Thm 3.3
				No	$O(n^{b+2})$	Cor 3.1
	b_I	Any	No	Any	Unary NPC	
					Thm 2.5	
		Yes	Yes	Any	$O(n \log n)$	
					Thm 3.2	
		No	Yes	Yes	$O(n \log n)$	Thm 3.3
				No	Unary NPC	Thm 3.4

which implies that jobs are lost only because of the input buffer constraint. A “No” in the $d_j = \infty$ column signifies that due dates are specified as part of problem input, and “Any” signifies either Yes or No. In the $p_j = 1$ column, “Yes” signifies that all processing times are equal, “No” signifies that processing times are arbitrary, and “Any” signifies either Yes or No. In the $s_j = 1$ column, “Yes,” “No,” and “Any” are similarly defined. However, “n.a.” appears in the $s_j = 1$ column when $b_I = 0$, which means that the storage requirement of jobs is not applicable. We use Unary (respectively, Binary) NPC to indicate that the equivalent recognition version of a problem is NP-complete with respect to a unary (resp., binary) encoding of the data (see Garey and Johnson 1979).

The *scheduling* of a job specifies a decision as to when the *processing* of that job takes place. A job that is not lost (or late) is processed *on time*. In a schedule, *inserted idle time* is machine idle time that can be removed, without altering the job sequence, by processing earlier the first job after the idle time. Since there is no advantage to having inserted idle time, only schedules without inserted idle time are considered.

We do not allow the machine to be used for storage. When a job is released, either it is discarded, or it is accepted for processing and must complete by its due date. In the latter case, the job either starts processing at its

release date or is stored in an input buffer. We refer to a job j as being *available* at time t if either $r_j = t$ or j is in the buffer at time t . Let $\mathcal{B}$ denote the set of all possible subsets of jobs that can be stored in the buffer simultaneously.

Let set $\mathcal{T}$ contain (possibly as a proper subset) all times t for which a job may be processed during the interval $[t, t + 1]$ in some feasible schedule. We construct $\mathcal{T}$ in Subroutine TGEN below. This subroutine allows for infinite buffer capacity and ignores due date considerations. Also, the processing time of each job is replaced by an upper bound, $p_{\max}$.

Subroutine TGEN

Initialization: Index the jobs such that $r_1 \leq \dots \leq r_n$. Set $r_{n+1} = \infty$, $p_{\max} = \max_{j \in N} \{p_j\}$, $k = 1$, $t = r_1$, and $\mathcal{T} = \emptyset$.
New Release Date: Set $l = \min\{j | j \in N \cup \{n+1\}, r_j > t\}$.

If $t + p_{\max} \leq r_l$, then go to Extension by Processing Time.

Extension by Release Date: Set $\mathcal{T} = \mathcal{T} \cup \{t, \dots, r_l - 1\}$, and $t = r_l$. Go to New Release Date.

Extension by Processing Time: Set $\mathcal{T} = \mathcal{T} \cup \{t, \dots, t + p_{\max} - 1\}$, $k = k + 1$, and $t = \max\{r_k, t + p_{\max}\}$.

If $k \leq n$, then go to New Release Date. Otherwise, terminate.

The following result, which is proved in detail in Hall et al. (1996), establishes an upper bound on $|\mathcal{T}|$.

Theorem 1.1. *Consider any input buffer scheduling problem for which $\mathcal{T}$ is obtained from TGEN. If a feasible schedule does not have inserted idle time, then processing occurs only during intervals $[t, t + 1]$ for $t \in \mathcal{T}$. Moreover, $|\mathcal{T}| \leq (2n - 1)p_{\max}$; for unit processing time problems, $|\mathcal{T}| \leq n$.*

Whenever $\mathcal{T}$ is used, we let $\eta = |\mathcal{T}|$ and $\mathcal{T} = \{\tau_1, \dots, \tau_\eta\}$, where $\tau_1 < \dots < \tau_\eta$. Since in some of our algorithms it is useful to work with completion times of unit time intervals, we define $\bar{\mathcal{T}} = \{t + 1 | t \in \mathcal{T}\}$, and let $\bar{\mathcal{T}} = \{\bar{\tau}_1, \dots, \bar{\tau}_\eta\}$, where $\bar{\tau}_1 < \dots < \bar{\tau}_\eta$.

2. WEIGHTED NUMBER OF LOST JOBS

Our analysis begins with problems involving the minimization of $\sum w_j U_j$ in which the buffer capacity is zero. Then, we consider fixed and arbitrary buffer capacities. The following result establishes that problems with zero buffer capacity and unit processing times are solvable in polynomial time.

Theorem 2.1. *An optimal schedule for $1|b_I = 0, r_j, p_j = 1|\sum w_j U_j$ and its restrictions can be found in $O(n)$ time, by processing at each release date t a job j , where $j = \arg\max\{w_i | i \in N, r_i = t\}$.*

Proof. Since $b_I = 0$, each on-time job is processed immediately upon release. We examine each job j in arbitrary order. If some job i with $w_i \geq w_j$ is already scheduled in $[r_j, r_j + 1]$, then job j is discarded. Otherwise, we schedule job j in this interval, possibly causing a previously scheduled job i with $w_i < w_j$ to be discarded. An interchange argument establishes that from the set of jobs with the same release date, one with largest weight should be on time. This proves the optimality of the algorithm. This algorithm requires constant time for each job. Consequently, the overall time complexity is $O(n)$. $\square$

An optimal schedule for $1|b_I = 0, r_j|\sum w_j U_j$ and its restrictions can be found in $O(n^2)$ time by an algorithm of Arkin and Silverberg (1987).

We now analyze fixed buffer capacity problems. First, we present a pseudo-polynomial time dynamic programming algorithm, DPW, for $1|b_I = b, r_j|\sum w_j U_j$. The states of DPW are (T, B) , for $T \in \bar{\mathcal{T}} \cup \{0\}$ and $B \in \mathcal{B}$. Any (partial) schedule corresponding to a state (T, B) has completion time T and buffer configuration B at time T . Each iteration of DPW increases a time index t by one and considers states for which $T = \bar{\tau}_t$. One job is appended to each partial schedule corresponding to state (T, B) , where $T = \bar{\tau}_t$ and $B \in \mathcal{B}$. Also, the buffer contents are updated to allow for the possible storage of newly released jobs. DPW recursively computes $\Omega(T, B)$, the maximum total weight of processed jobs among all partial schedules generated thus far that achieve the state (T, B) . A completed schedule has $B = \emptyset$. Thus, the maximum total weight of processed jobs in an optimal schedule is equal to $\max_{T \in \bar{\mathcal{T}}} \{\Omega(T, \emptyset)\}$.

Algorithm DPW

Initialization: Call TGEN, and compute $\bar{\mathcal{T}}$. Set $\Omega(T, B) = -\infty$ for all $T \in \bar{\mathcal{T}} \cup \{0\}$ and $B \in \mathcal{B}$. Set $t = 0$, $T = 0$ and $\Omega(T, \emptyset) = 0$.

Job Scheduling: For each $B \in \mathcal{B}$ where $\Omega(T, B) > -\infty$, and each $k \in \{j | j \in N, r_j \geq T\} \cup B$ where $T + p_k \leq d_k$ and $\max\{T, r_k\} + p_k \in \bar{\mathcal{T}}$:

- Set $T' = \max\{T, r_k\} + p_k$, $V' = \Omega(T, B) + w_k$ and $B' = B \setminus \{k\}$.
- For each $B_1 \subseteq \{j | j \in N, T \leq r_j < T', j \neq k\}$ such that $\sum_{i \in B' \cup B_1} s_i \leq b$ and $V' > \Omega(T', B' \cup B_1)$, set $\Omega(T', B' \cup B_1) = V'$.

Termination Test: If $t < |\bar{\mathcal{T}}| - 1$, then set $t = t + 1$ and $T = \bar{\tau}_t$, and go to Job Scheduling.

Output: Compute $\max_{T \in \bar{\mathcal{T}}} \{\Omega(T, \emptyset)\}$. Backtrack to find the corresponding schedule. Output the schedule and the total weight of scheduled jobs, and terminate.

Given a partial schedule, the next job to be scheduled in the Job Scheduling step, job k , is either currently in the buffer, or is released at time T or later. The jobs of B_1 , which become available for processing in the interval $[T, T' - 1]$, are candidates for insertion in the buffer. Job k is not in the buffer at any time in this interval. Therefore, to ensure that the buffer capacity constraint is satisfied, it is sufficient to check that the capacity is not exceeded just before the processing of job k finishes.

There is an alternative pseudo-polynomial time dynamic programming algorithm, DPT, for $1|b_I = b, r_j|\sum w_j U_j$. We obtain DPT from DPW by switching the value, V , stored in $\Omega(T, B)$, with the state variable T . Thus, in DPT, $\Omega(V, B)$ denotes the completion time of a schedule where the processed jobs have total weight V , the jobs of B are in the buffer at the completion of the schedule, and which has minimum completion time among such schedules generated thus far. The recursions for Algorithms DPT and DPW are similar.

To show that DPW finds an optimal schedule for $1|b_I = b, r_j|\sum w_j U_j$, we need a preliminary result.

Lemma 2.1. *Suppose that DPW finds two partial schedules that reach the same state (T, B) with total weight of scheduled jobs V_1 and V_2 , respectively. If $V_1 \geq V_2$, then the second schedule can be discarded in the search for an optimal schedule.*

Proof. Follows from the observation that any feasible continuation of the schedule with weight V_2 is also feasible for the schedule with weight V_1 . $\square$

Theorem 2.2. *DPW and DPT generate optimal schedules for $1|b_I = b, r_j|\sum w_j U_j$ and its restrictions. DPW requires $O(n^{b+2}p_{\max})$ time, while DPT requires $O(n^{b+2}w_{\max})$ time.*

Proof. Any job k that is not considered in the Job Scheduling step of DPW is either already lost or cannot complete on time. Also, any value $\Omega(T', B' \cup B_1)$ that is not updated satisfies the conditions of Lemma 2.1. Thus, DPW finds an optimal schedule.

The time complexity of DPW is derived next. The maximum number of possible buffer configurations is $O(n^b)$. Since Theorem 1.1 shows that there are $O(np_{\max})$ choices for T , the Initialization step requires $O(n^{b+1}p_{\max})$ time. Consider the states (T, B) for which the total buffer storage requirement is $\sum_{i \in B} s_i = b'$ for some b' where $0 \leq b' \leq b$. Assume initially that T and b' are fixed. Assume that some fixed job k is to be processed next. Suppose first that $k \notin B$. There are $O(n^{b'})$ states where $\sum_{i \in B} s_i = b'$, and $O(n^{b-b'})$ choices for B_1 . Therefore, when $k \notin B$, the computational requirement of the Job Scheduling step is $O(n^b)$. Alternatively, suppose that $k \in B$. Since the storage requirement of job k is s_k , there are $O(n^{b'-s_k})$ states, each of which yields $O(n^{b-b'+s_k})$ choices for B_1 . Therefore, when $k \in B$ for fixed k , T and b' , the Job Scheduling step again requires $O(n^b)$ time. From Theorem 1.1, there are $O(np_{\max})$ choices for T . There are $O(n)$ choices for k , and a constant number of choices for b' . Thus, the overall time complexity for the Job Scheduling step is $O(n^{b+2}p_{\max})$. From Theorem 1.1, the Output step requires $O(np_{\max})$ time. Therefore, the overall time requirement of DPW is $O(n^{b+2}p_{\max})$, which completes the proof for DPW.

For DPT, we can establish a result analogous to Lemma 2.1. Thus, we can show that DPT finds an optimal schedule in $O(n^{b+2}w_{\max})$ time. $\square$

Nawijn et al. (1994) show that the recognition version of $1|b_I = b, r_j, d_j = \infty, s_j = 1|\sum w_j U_j$ is binary NP-complete. This completes the complexity analysis for problems with arbitrary processing times and fixed buffer capacity. We now consider unit processing time problems.

Corollary 2.1. *DPW generates an optimal schedule for $1|b_I = b, r_j, p_j = 1|\sum w_j U_j$ and its restrictions in $O(n^{b+2})$ time.*

Proof. The result follows from Theorem 2.2 and the observation that $p_{\max} = 1$. $\square$

For the case of unit processing times and unit storage requirements, we propose a more efficient algorithm, LWR, for $1|b_I, r_j, d_j = \infty, p_j = 1, s_j = 1|\sum w_j U_j$. In LWR, an available job with largest weight is the one processed next. Also, if some jobs must be discarded to prevent buffer overflow, then jobs with largest weights are the ones retained.

Algorithm LWR

Initialization: Index the jobs such that $r_1 \leq \dots \leq r_n$. Set $k = 1$, $t = r_1$, and $B = \emptyset$.

Adjusting Buffer: Set $l = \max\{i | i \in N, r_k = \dots = r_i\}$, $B = B \cup \{k, \dots, l\}$, and $k = l + 1$. If $|B| > b_I + 1$, then let $B' \subseteq B$ be a set of $b_I + 1$ jobs with largest weights. Set $B = B'$.

Largest Weight Scheduling: Process job j during the interval $[t, t + 1]$, where $j = \operatorname{argmax}_{i \in B} \{w_i\}$. Set $B = B \setminus \{j\}$ and $t = t + 1$.

Termination Test: If $k \leq n$ and $t = r_k$, then go to Adjusting Buffer. If $B = \emptyset$ and $k \leq n$, then set $t = r_k$ and go to Adjusting Buffer. If $B \neq \emptyset$, then go to Largest Weight Scheduling.

Output: Output the schedule and the total weight of scheduled jobs, and terminate.

Theorem 2.3. *LWR generates an optimal schedule for $1|b_I, r_j, d_j = \infty, p_j = 1, s_j = 1|\sum w_j U_j$ and its restrictions in $O(n \log n)$ time.*

Proof. Optimality is established by an interchange argument. Using the weight as a key to maintain the jobs in the buffer in a heap (Kruse 1987), and indexing the jobs in release date order, require $O(n \log n)$ time. Thus, the overall time requirement is $O(n \log n)$. $\square$

A proof of the following result appears in Hall et al. (1996).

Theorem 2.4. *The recognition version of $1|b_I, r_j, d_j = \infty, p_j = 1|\sum w_j U_j$ and its generalizations are unary NP-complete.*

We now consider problems with arbitrary processing times but unit storage requirements.

Theorem 2.5. *The recognition version of $1|b_I, r_j, d_j = \infty, s_j = 1|\sum U_j$ and its generalizations are unary NP-complete.*

Proof. Consider the output buffer problem $1|b_O, e_j = d'_j, s_j = 1|\sum U'_j$ discussed by Hall et al. (1998), with n' jobs, where b_O denotes an output buffer with arbitrary capacity, e_j is the earliest dispatch time to a customer and p'_j is the processing time of job j . Given an instance of $1|b_O, e_j = d'_j, s_j = 1|\sum U'_j$, we construct an instance of $1|b_I, r_j, d_j = \infty, s_j = 1|\sum U_j$ by setting $n = n'$, $b_I = b_O$, $r_j = \max_{i \in N} \{d'_i\} - d'_j$ and $p_j = p'_j$ for $j = 1, \dots, n$. A set of jobs is on time for a schedule in one problem if and only if the same set of jobs is on time for the schedule with jobs in reverse order in the other problem. Thus, the two problems have $U_j = U'_j$ for $j = 1, \dots, n$. Since Hall et al. (1998) show that the recognition version of $1|b_O, e_j = d'_j, s_j = 1|\sum U'_j$ is unary NP-complete, then so is the recognition version of $1|b_I, r_j, d_j = \infty, s_j = 1|\sum U_j$. $\square$

We note that this equivalence between output and input buffer problems, which we call *reversibility*, can be used to infer algorithms as well as complexity results.

Next, problem $1|b_I, r_j, p_j = 1, s_j = 1|\sum w_j U_j$ is considered. Let $\{\rho_1, \dots, \rho_q\}$ denote the set of *distinct* release dates, where $\rho_1 < \dots < \rho_q$. Also, let $\rho_{q+1} = \infty$, and let $T_j = \mathcal{T} \cap \{r_j, \dots, d_j - 1\}$ for $j \in N$, where $\mathcal{T}$ is found by applying TGEN of Section 1. For $k \in K = \{1, \dots, q\}$, let $M_k = \{j | j \in N, r_j = \rho_k\}$ and $R_k = \mathcal{T} \cap \{\rho_k + 1, \dots, \rho_{k+1}\}$. Further, let $M_{q+1} = \emptyset$ and $R_0 = \{\rho_1\}$.

For $1|b_I, r_j, p_j = 1, s_j = 1|\sum w_j U_j$, we construct and solve an equivalent minimum cost network flow problem. By defining variables

$$x_{jt} = \begin{cases} 1 & \text{if job } j \text{ is processed during} \\ & \text{the interval } [t, t + 1], \quad j \in N, t \in T_j, \\ 0 & \text{otherwise,} \end{cases}$$

$$u_j = \begin{cases} 1 & \text{if job } j \text{ is lost,} \\ 0 & \text{otherwise,} \end{cases} \quad j \in N,$$

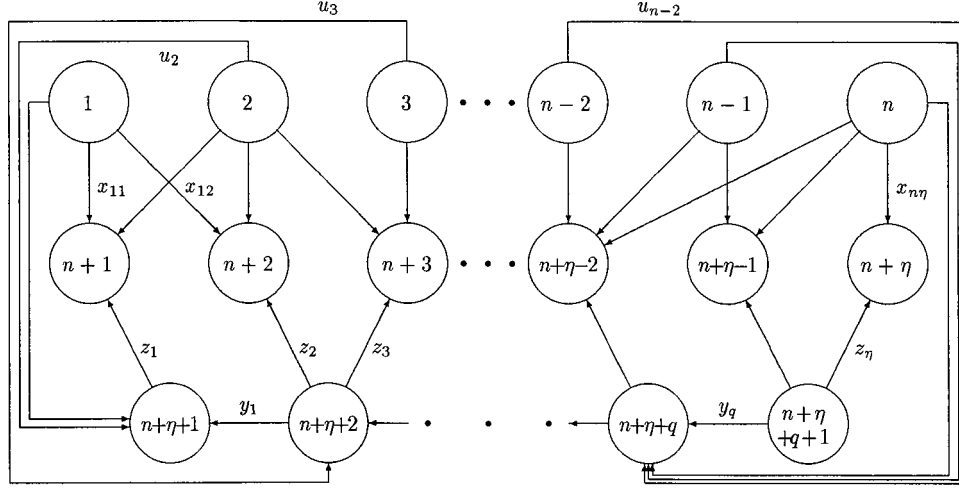


Figure 1. Example network formulation of problem $1|b_t, r_j, p_j = 1, s_j = 1|\sum w_j U_j$.

$$z_t = \begin{cases} 1 & \text{if the machine is idle} \\ & \text{during the interval } [t, t+1], \quad t \in \mathcal{T}, \\ 0 & \text{otherwise,} \end{cases}$$

y_k = used buffer space immediately after time ρ_k , $k \in K$,

we obtain the formulation

$$\begin{aligned} & \text{minimize } \sum_{j \in N} w_j u_j \\ & \text{subject to } \sum_{t \in T_j} x_{jt} + u_j = 1, \quad j \in N, \end{aligned} \quad (1)$$

$$\sum_{j \in \{i|i \in N, t \in T_i\}} x_{jt} + z_t = 1, \quad t \in \mathcal{T}, \quad (2)$$

$$\begin{aligned} y_k - y_{k-1} + \sum_{j \in M_k} u_j - \sum_{t \in R_{k-1}} z_t \\ = |M_k| - |R_{k-1}|, \quad k \in K \cup \{q+1\}, \end{aligned} \quad (3)$$

where $x_{jt} \in \{0, 1\}$, $j \in N$, $t \in T_j$; $u_j \in \{0, 1\}$, $j \in N$; $z_t \in \{0, 1\}$, $t \in \mathcal{T}$; $y_k \in \{0, \dots, b_k\}$, $k \in K$; and $y_k = 0$, $k \in \{0, q+1\}$. Constraints (1) ensure that each job is either processed in some time interval or is lost. Similarly, constraints (2) specify that each time interval is either used to process a job or is idle. The used buffer space is updated at each release date in (3). Processed jobs that have start times in R_{k-1} decrease the used buffer space by $|R_{k-1}| - \sum_{t \in R_{k-1}} z_t$ for $k \in K \cup \{q+1\}$. Also, at time ρ_k , arriving jobs that are not lost increase used buffer space by $|M_k| - \sum_{j \in M_k} u_j$.

The constraint coefficients define the node-arc incidence matrix of a network $G = (V, A)$, where $V = \{v_1, \dots, v_{n+\eta+q+1}\}$ denotes the node set, A denotes the directed arc set, and where $\eta = |\mathcal{T}|$. Nodes $v_1, \dots, v_n$ correspond to constraints (1), nodes $v_{n+1}, \dots, v_{n+\eta}$ correspond to constraints (2), and nodes $v_{n+\eta+1}, \dots, v_{n+\eta+q+1}$ correspond to constraints (3). The set A consists of four types of directed arcs. For each variable x_{jt} , we have a zero-cost arc $(v_j, v_{n+t'})$, where t' satisfies $\tau_{t'} = t$ (t' is the index of the time in $\mathcal{T}$ which corresponds to the start of the interval $[t, t+1]$). For each variable u_j , there is an arc $(v_j, v_{n+\eta+k})$, where k satisfies $r_j = \rho_k$, with cost w_j . For each variable z_t , there is a zero-cost arc $(v_{n+\eta+k}, v_{n+t'})$, where k

satisfies $t \in R_{k-1}$ and t' satisfies $\tau_{t'} = t$. Finally, each variable y_k generates a zero-cost arc $(v_{n+\eta+k+1}, v_{n+\eta+k})$, for $k \in K$, which has capacity b_k .

The demand at node v_i is -1 for $i = 1, \dots, n$; is 1 for $i = n+1, \dots, n+\eta$; and is $|M_{i-n-\eta}| - |R_{i-n-\eta-1}|$ for $i = n+\eta+1, \dots, n+\eta+q+1$. Figure 1 illustrates network G , where several arcs in G have been labeled with the variables that represent the flow along them. The following result shows that G can be used to solve problem $1|b_t, r_j, p_j = 1, s_j = 1|\sum w_j U_j$.

Theorem 2.6. An optimal schedule for $1|b_t, r_j, p_j = 1, s_j = 1|\sum w_j U_j$ can be found in $O(n^3 \log n)$ time.

Proof. Since Theorem 1.1 shows that $\eta \leq n$, the formulation above provides a representation of problem $1|b_t, r_j, p_j = 1, s_j = 1|\sum w_j U_j$ as a minimum cost flow problem on a network $G = (V, A)$ with $|V| = O(n)$ nodes and $|A| = O(n^2)$ arcs. Moreover, there exists an optimal solution with integral flows on all arcs. The enhanced capacity scaling algorithm of Orlin (1988) finds a minimum cost flow in G in $O(|V| \log |V| (|A| + |V| \log |V|))$ time. Thus, the time complexity is $O(n^3 \log n)$. $\square$

3. NUMBER OF LOST JOBS

In this section, we analyze problems where the objective is to minimize $\sum U_j$.

Theorem 2.1 shows that $1|b_t = 0, r_j, p_j = 1|\sum U_j$ is solvable in $O(n)$ time. For problems with zero buffer capacity and arbitrary processing times, the absence of buffer space implies that any job which is not lost must be processed during the interval $[r_j, r_j + p_j]$. Hall et al. (1998) derive an $O(n \log n)$ algorithm, GREEDY-EDD, for the finite capacity output buffer problem $1|b_o = 0, e_j = d_j|\sum U_j$, where b_o and e_j are defined in the proof of Theorem 2.5. In this algorithm, any job j that is on time must be processed during the interval $[e_j - p_j, e_j]$. The algorithm schedules a job with smallest e_j value from the set of unscheduled jobs that can complete processing by their due dates. By setting $e_j = r_j + p_j$, we can convert

any instance of $1|b_I = 0, r_j|\sum U_j$ into an equivalent instance of $1|b_O = 0, e_j = d_j|\sum U_j$. Thus, we have the following result.

Theorem 3.1. *GREEDY-EDD generates an optimal schedule for $1|b_I = 0, r_j|\sum U_j$ and its restrictions in $O(n \log n)$ time.*

For fixed buffer capacity problems, DPT provides an optimal schedule in polynomial time.

Corollary 3.1. *DPT generates an optimal schedule for $1|b_I = b, r_j|\sum U_j$ and its restrictions in $O(n^{b+2})$ time.*

Proof. The results follow from Theorem 2.2 and the observation that $w_{\max} = 1$. $\square$

For unit processing time problems, we construct algorithms that are similar to LWR and are applicable to problems with both fixed and arbitrary buffer capacities. For $1|b_I, r_j, d_j = \infty, p_j = 1|\sum U_j$, we propose Algorithm SSR. In SSR, an available job with largest storage requirement is processed next. Also, when some jobs must be lost because of insufficient buffer capacity, the jobs with smallest storage requirements are retained.

Algorithm SSR

Identical to LWR, except that the second part of the Adjusting Buffer step of LWR is replaced by:

If $\sum_{i \in B} s_i > b_I + \max_{i \in B} \{s_i\}$, then let $B' \subseteq B$ be a maximum cardinality set of jobs such that $\sum_{i \in B'} s_i - \max_{i \in B'} \{s_i\} \leq b_I$, and $s_i \leq s_j$ for all $i \in B'$ and $j \in B \setminus B'$. Set $B = B'$.

Also, in the Largest Weight Scheduling step, we process job j during the interval $[t, t + 1]$, where $j = \arg \max_{i \in B} \{s_i\}$.

Theorem 3.2. *SSR generates an optimal schedule for $1|b_I, r_j, d_j = \infty, p_j = 1|\sum U_j$ and its restrictions in $O(n \log n)$ time.*

Proof. Similar to the proof of Theorem 2.3. $\square$

For $1|b_I > 0, r_j, p_j = 1, s_j = 1|\sum U_j$, we propose Algorithm LDR. In LDR, the available jobs that can complete by their due dates are processed in EDD order. Also, when some jobs must be lost because of insufficient buffer capacity, the jobs with largest due dates are retained.

Algorithm LDR

Identical to LWR, except that the second part of the Adjusting Buffer step of LWR is replaced by:

If $|B| > b_I + 1$, then let $B' \subseteq B$ be a set of $b_I + 1$ jobs with largest due dates. Set $B = B'$.

Also, in the Largest Weight Scheduling step, we process job j during the interval $[t, t + 1]$ where $j = \arg \min_{i \in B} \{d_i | d_i > t\}$.

Theorem 3.3. *LDR generates an optimal schedule for $1|b_I, r_j, p_j = 1, s_j = 1|\sum U_j$ and its restrictions in $O(n \log n)$ time.*

Proof. Similar to the proof of Theorem 2.3. $\square$

For arbitrary capacity buffer problems, Theorem 2.5 establishes that several problems are unary NP-complete. A proof of the following result for unit processing time problems appears in Hall et al. (1996).

Theorem 3.4. *The recognition version of $1|b_I, r_j, p_j = 1|\sum U_j$ is unary NP-complete.*

4. CONCLUDING REMARKS

We describe algorithms and computational complexity results for several scheduling problems with finite capacity input buffers. In view of increasing use of modern manufacturing principles that attempt to limit buffer space, our work has practical implications for the design of manufacturing systems. In particular, we identify circumstances under which the limitation of input buffer capacity can simplify or complicate scheduling decisions. Future research needs to improve the order of complexity of the polynomial time, enumerative algorithms to make them more practical. It might be possible to apply reversibility arguments of the type used in the proof of Theorem 2.5 to extend our algorithms and NP-completeness results to other problems. Also, bounds, heuristics, and computationally efficient procedures are needed for the NP-hard problems.

ACKNOWLEDGMENTS

The authors would like to thank Professor W. M. Nawijn for suggesting the reversibility idea that is used to prove Theorem 2.5. This research is supported in part by the Summer Fellowship Program (Fisher College of Business, The Ohio State University) and a grant from The Royal Society.

REFERENCES

- ARKIN, E. M. AND E. B. SILVERBERG. 1987. Scheduling Jobs with Fixed Start and End Times. *Discrete Applied Mathematics*, **18**, 1–8.
- GAREY, M. R. AND D. S. JOHNSON. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, San Francisco, CA.
- GRAHAM, R. L., E. L. LAWLER, J. K. LENSTRA, AND A. H. G. RINNOOY KAN. 1979. Optimization and Approximation in Deterministic Machine Scheduling: A Survey. *Ann. Discrete Math.* **5**, 287–326.
- HALL, N. G., M. E. POSNER, AND C. N. POTTS. 1996. Scheduling with Finite Capacity Input Buffers. Technical Report, Industrial Engineering, The Ohio State University, Columbus, Ohio.
- HALL, N. G., M. E. POSNER, AND C. N. POTTS. 1997. Preemptive Scheduling with Finite Capacity Input Buffers. *Ann. O. R.*, **70**, 399–413.
- HALL, N. G., M. E. POSNER, AND C. N. POTTS. 1998. Scheduling with Finite Capacity Output Buffers. *Opns. Res.* **46**, this issue.
- KRUSE, R. L. 1987. *Data Structures and Program Design*, Second Edition. Prentice-Hall, Englewood Cliffs, NJ.
- MCCLAINE, J. O. AND L. J. THOMAS. 1985. *Operations Management*, Second Edition. Prentice-Hall, Englewood Cliffs, NJ.
- NAWIJN, W. M. 1992. Minimum Loss Scheduling Problems. *European J. Opnl. Res.* **56**, 364–369.
- NAWIJN, W. M., W. KERN, AND S. BAAS. 1994. Scheduling Jobs with Release Times on a Machine with Finite Storage. *European J. Opnl. Res.* **74**, 120–127.
- ORLIN, J. B. 1988. A Faster Strongly Polynomial Minimum Cost Flow Algorithm. In *Proceedings 20th ACM Symposium on Theory of Computing*, 377–387.